Matrimony Sales Analysis

Abstract

This project analyzes matrimony platform data using SQL to understand user demographics, partner preferences, and sales performance. The insights support better business decision-making.

Introduction

The matrimony industry relies on user data and preferences. This project analyzes customer and sales data to identify trends and improve matchmaking efficiency.

Objectives

- Analyze user demographics
- Study partner preferences
- Evaluate sales performance
- Identify top customers
- Understand revenue trends

Database Design

Tables used: Users, Education Details, Family Details, Location Details, Sales, Partner Preferences.

Data Description

Users: Personal details like name, gender, DOB.
Sales: Membership type, amount paid, payment date.
Preferences: Age, height, religion, location.

SQL Techniques Used

Joins, Aggregations, Subqueries, CTEs, Views, Window Functions.

Key Analysis

Revenue Analysis, Customer Analysis, Preference Analysis.

Conclusion

This project demonstrates how SQL can be used to extract meaningful insights from matrimony data and improve business strategies.

SQL Quary

INPUT

#Data Base Creation:

```
Create database matrimonysalesanalysis;
```

#Using Database

```
Use matrimonysalesanalysis ;
```

#Users table creation

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100),
    gender ENUM('Male', 'Female'),
    date_of_birth DATE,
    mother_tongue VARCHAR(50),
    religion VARCHAR(50),
    caste VARCHAR(50),
    marital_status ENUM('Never Married', 'Divorced', 'Widowed'),
    mobile_number VARCHAR(15),
    email VARCHAR(100),
    location_id INT,
    created_at DATETIME
);
```

#Inserting data into user table

```
INSERT INTO users (
    user_id, full_name, gender, date_of_birth, mother_tongue, religion, caste, marital_status, mobile_number,
email, location_id, created_at
) VALUES
('1000', 'Ravi Kumar', 'Male', '1990-05-10', 'Tamil', 'Hindu', 'Iyer', 'Never Married', '9876543210',
'ravi.kumar@example.com', '51000', NOW()),
('1001', 'Priya Sharma', 'Female', '1992-07-15', 'Telugu', 'Hindu', 'Kamma', 'Never Married', '9876543211',
'priya.sharma@example.com', '51001', NOW()),
('1002', 'Arun Raj', 'Male', '1988-12-01', 'Tamil', 'Christian', 'Nadar', 'Divorced', '9876543212',
'arun.raj@example.com', '51002', NOW()),
```

```
('1003', 'Meena George', 'Female', '1991-09-20', 'Malayalam', 'Christian', 'Syrian', 'Never Married', '9876543213',
'meena.george@example.com', '51003', NOW()),

('1004', 'Vikram Desai', 'Male', '1987-03-05', 'Gujarati', 'Hindu', 'Patel', 'Widowed', '9876543214',
'vikram.desai@example.com', '51004', NOW()),

('1005', 'Anjali Rao', 'Female', '1993-11-11', 'Kannada', 'Hindu', 'Brahmin', 'Never Married', '9876543215',
'anjali.rao@example.com', '51005', NOW()),

('1006', 'Karthik M', 'Male', '1990-01-25', 'Telugu', 'Hindu', 'Reddy', 'Never Married', '9876543216',
'karthik.m@example.com', '51006', NOW()),

('1007', 'Divya S', 'Female', '1989-06-30', 'Tamil', 'Hindu', 'Vellalar', 'Divorced', '9876543217',
'divya.s@example.com', '51007', NOW()),

('1008', 'Rahul Verma', 'Male', '1992-08-08', 'Hindi', 'Hindu', 'Kayastha', 'Never Married', '9876543218',
'rahul.verma@example.com', '51008', NOW()),

('1009', 'Sneha Iyer', 'Female', '1990-10-12', 'Tamil', 'Hindu', 'Iyer', 'Never Married', '9876543219',
'sneha.iyer@example.com', '51009', NOW()),

('1010', 'Amitabh Joshi', 'Male', '1985-04-14', 'Marathi', 'Hindu', 'Deshastha', 'Divorced', '9876543220',
'amitabh.joshi@example.com', '51010', NOW()),

('1011', 'Lakshmi Menon', 'Female', '1994-03-18', 'Malayalam', 'Hindu', 'Nair', 'Never Married', '9876543221',
'lakshmi.menon@example.com', '51011', NOW()),

('1012', 'Suresh Babu', 'Male', '1989-09-25', 'Telugu', 'Hindu', 'Kapu', 'Never Married', '9876543222',
'suresh.babu@example.com', '51012', NOW()),

('1013', 'Nisha Reddy', 'Female', '1991-12-10', 'Telugu', 'Hindu', 'Reddy', 'Never Married', '9876543223',
'nisha.reddy@example.com', '51013', NOW()),

('1014', 'Manoj Mehta', 'Male', '1986-06-06', 'Gujarati', 'Jain', 'Shwetambar', 'Widowed', '9876543224',
'manoj.mehta@example.com', '51014', NOW()),

('1015', 'Sangeetha D', 'Female', '1995-01-01', 'Tamil', 'Hindu', 'Chettiar', 'Never Married', '9876543225',
'sangeetha.d@example.com', '51015', NOW()),

('1016', 'Abhishek Nair', 'Male', '1990-07-07', 'Malayalam', 'Hindu', 'Nair', 'Never Married', '9876543226',
'abhishek.nair@example.com', '51016', NOW()),

('1017', 'Pooja Agarwal', 'Female', '1992-02-20', 'Hindi', 'Hindu', 'Agarwal', 'Never Married', '9876543227',
'pooja.agarwal@example.com', '51017', NOW()),

('1018', 'Rohit Jain', 'Male', '1988-10-30', 'Hindi', 'Jain', 'Digambar', 'Divorced', '9876543228',
'rohit.jain@example.com', '51018', NOW()),

('1019', 'Bhavana K', 'Female', '1990-08-22', 'Kannada', 'Hindu', 'Brahmin', 'Never Married', '9876543229',
'bhavana.k@example.com', '51019', NOW());

# Create education table

CREATE TABLE education_details (
```

```
    education_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    highest_qualification VARCHAR(100),
    occupation VARCHAR(100),
    annual_income DECIMAL(10,2),
    institution VARCHAR(100),
    graduation_year INT
);

#Inserting data into education table
INSERT INTO education_details (
education_id, user_id, highest_qualification, occupation, annual_income, institution, graduation_year
) VALUES
('31000', '1000', 'B.Tech', 'Software Engineer', 1200000.00, 'Anna University', 2011),
('31001', '1001', 'MBA', 'Manager', 1500000.00, 'IIM Ahmedabad', 2015),
('31002', '1002', 'B.Com', 'Accountant', 800000.00, 'Mumbai University', 2009),
('31003', '1003', 'M.Sc', 'Analyst', 950000.00, 'Kerala University', 2014),
('31004', '1004', 'M.Tech', 'DevOps Engineer', 1300000.00, 'IIT Hyderabad', 2012),
('31005', '1005', 'B.Sc', 'Teacher', 600000.00, 'Delhi University', 2017),
('31006', '1006', 'BBA', 'Sales Manager', 700000.00, 'Guru Nanak Dev University', 2013),
('31007', '1007', 'MBA', 'HR Manager', 800000.00, 'Xavier Labour Relations Institute', 2016),
('31008', '1008', 'MCA', 'Software Developer', 900000.00, 'University of Pune', 2014),
('31009', '1009', 'B.A. Economics', 'Economist', 650000.00, 'Aligarh Muslim University', 2010),
('31010', '1010', 'B.Tech', 'Civil Engineer', 850000.00, 'MIT Pune', 2012),
('31011', '1011', 'MBA', 'Project Manager', 1200000.00, 'IIM Bangalore', 2014),
('31012', '1012', 'M.Sc', 'Biologist', 700000.00, 'Jawaharlal Nehru University', 2015),
('31013', '1013', 'B.A. Sociology', 'Social Worker', 400000.00, 'Tata Institute of Social Sciences', 2008),
('31014', '1014', 'B.Tech', 'Network Engineer', 1100000.00, 'BITS Pilani', 2013),
('31015', '1015', 'B.Com', 'Business Analyst', 950000.00, 'Delhi University', 2012),
```

```
('31016', '1016', 'M.Tech', 'AI Researcher', 1400000.00, 'IIT Bombay', 2016),
('31017', '1017', 'M.A. Psychology', 'Clinical Psychologist', 750000.00, 'University of Delhi', 2014),
('31018', '1018', 'B.Tech', 'Software Developer', 950000.00, 'VIT Vellore', 2013),
('31019', '1019', 'M.Sc', 'Statistician', 800000.00, 'University of Calcutta', 2015);

# Family_details table creation
Create Table family_details(
family_id INT AUTO_INCREMENT PRIMARY KEY,
user_id int,
father_name varchar(100),
father_occupation varchar(100),
mother_name varchar(100),
mother_occupation varchar(100),
siblings int,
foreign key (user_id) REFERENCES users(user_id)
);

# Inseting data into family_detail table
INSERT INTO family_details (
family_id, user_id, father_name, father_occupation, mother_name, mother_occupation, siblings
) VALUES
('41000', '1000', 'V. Kumar', 'Businessman', 'S. Priya', 'Homemaker', 2),
('41001', '1001', 'R. Sharma', 'Doctor', 'M. Sharma', 'Teacher', 1),
('41002', '1002', 'M. D''Souza', 'Engineer', 'F. D''Souza', 'Housewife', 3),
('41003', '1003', 'S. Menon', 'Bank Manager', 'K. Menon', 'Teacher', 1),
('41004', '1004', 'R. Reddy', 'Farmer', 'S. Reddy', 'Housewife', 4),
('41005', '1005', 'R. Yadav', 'Teacher', 'S. Yadav', 'Housewife', 2),
('41006', '1006', 'B. Singh', 'Army Officer', 'P. Kaur', 'Nurse', 3),
('41007', '1007', 'M. Patel', 'Engineer', 'S. Patel', 'Homemaker', 2),
```

```
('41008', '1008', 'A. Rao', 'Doctor', 'R. Rao', 'Teacher', 0),
('41009', '1009', 'S. Khan', 'Teacher', 'Z. Khan', 'Homemaker', 1),
('41010', '1010', 'V. Sharma', 'Lawyer', 'R. Sharma', 'Doctor', 3),
('41011', '1011', 'K. Verma', 'Businessman', 'S. Verma', 'Housewife', 2),
('41012', '1012', 'R. Joshi', 'Politician', 'P. Joshi', 'Teacher', 1),
('41013', '1013', 'A. Joshi', 'Engineer', 'L. Joshi', 'Nurse', 2),
('41014', '1014', 'V. Pandey', 'Scientist', 'M. Pandey', 'Teacher', 3),
('41015', '1015', 'R. Kumar', 'Software Developer', 'V. Kumar', 'Nurse', 2),
('41016', '1016', 'S. Kaur', 'Teacher', 'G. Kaur', 'Housewife', 0),
('41017', '1017', 'R. Reddy', 'Mechanic', 'K. Reddy', 'Housewife', 4),
('41018', '1018', 'A. Menon', 'Engineer', 'M. Menon', 'Doctor', 1),
('41019', '1019', 'J. Sharma', 'Software Engineer', 'S. Sharma', 'Teacher', 2);

# location table creation
CREATE TABLE location_details (
   location_id INT AUTO_INCREMENT PRIMARY KEY,
   user_id INT,
   city VARCHAR(100),
   state VARCHAR(100),
   country VARCHAR(100),
   foreign key (user_id) REFERENCES users(user_id)
);

-- Inserting data into location table --
INSERT INTO location_details (
location_id, user_id, city, state, country
) VALUES
('51000', '1000', 'Chennai', 'Tamil Nadu', 'India'),
('51001', '1001', 'Delhi', 'Delhi', 'India'),
```

```
('51002', '1002', 'Mumbai', 'Maharashtra', 'India'),
('51003', '1003', 'Kochi', 'Kerala', 'India'),
('51004', '1004', 'Hyderabad', 'Telangana', 'India'),
('51005', '1005', 'Lucknow', 'Uttar Pradesh', 'India'),
('51006', '1006', 'Amritsar', 'Punjab', 'India'),
('51007', '1007', 'Ahmedabad', 'Gujarat', 'India'),
('51008', '1008', 'Bangalore', 'Karnataka', 'India'),
('51009', '1009', 'Kolkata', 'West Bengal', 'India'),
('51010', '1010', 'Jaipur', 'Rajasthan', 'India'),
('51011', '1011', 'Chandigarh', 'Chandigarh', 'India'),
('51012', '1012', 'Noida', 'Uttar Pradesh', 'India'),
('51013', '1013', 'Pune', 'Maharashtra', 'India'),
('51014', '1014', 'Coimbatore', 'Tamil Nadu', 'India'),
('51015', '1015', 'Indore', 'Madhya Pradesh', 'India'),
('51016', '1016', 'Surat', 'Gujarat', 'India'),
('51017', '1017', 'Visakhapatnam', 'Andhra Pradesh', 'India'),
('51018', '1018', 'Patna', 'Bihar', 'India'),
('51019', '1019', 'Vadodara', 'Gujarat', 'India');

# Create product_detais table
CREATE TABLE sales (
    sale_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    product_type ENUM('Silver', 'Gold', 'Platinum'),
    amount_paid DECIMAL(10,2),
    payment_date DATE,
    foreign key (user_id) REFERENCES users(user_id)
);
```

#Insert data into sales table

```
INSERT INTO sales (sale_id, user_id, product_type, amount_paid, payment_date) VALUES
(61000, 1000, 'Silver', 1999.00, '2023-01-15'),
(61001, 1001, 'Gold', 4999.00, '2023-02-10'),
(61002, 1002, 'Platinum', 7999.00, '2023-03-05'),
(61003, 1003, 'silver', 9999.00, '2023-03-20'),
(61004, 1004, 'Gold', 4999.00, '2023-04-18'),
(61005, 1005, 'Silver', 1999.00, '2023-05-01'),
(61006, 1006, 'Platinum', 7999.00, '2023-06-12'),
(61007, 1007, 'Gold', 9999.00, '2023-07-23'),
(61008, 1008, 'Gold', 4999.00, '2023-08-14'),
(61009, 1009, 'Silver', 1999.00, '2023-09-09'),
(61010, 1010, 'Platinum', 7999.00, '2023-10-21'),
(61011, 1011, 'Platinum', 9999.00, '2023-11-05'),
(61012, 1012, 'Silver', 1999.00, '2023-11-30'),
(61013, 1013, 'Gold', 4999.00, '2023-12-10'),
(61014, 1014, 'Platinum', 7999.00, '2024-01-04'),
(61015, 1015, 'Gold', 9999.00, '2024-01-20'),
(61016, 1016, 'Silver', 1999.00, '2024-02-08'),
(61017, 1017, 'Gold', 4999.00, '2024-02-25'),
(61018, 1018, 'Platinum', 7999.00, '2024-03-11'),
(61019, 1019, 'Silver', 9999.00, '2024-04-01');
```

#Ppartner_preference

```
CREATE TABLE partner_preferences (
    preference_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    preferred_age_from INT,
    preferred_age_to INT,
```

```
    preferred_height_from DECIMAL(3,2),
    preferred_height_to DECIMAL(3,2),
    preferred_mother_tongue VARCHAR(50),
    preferred_religion VARCHAR(50),
    preferred_caste VARCHAR(50),
    preferred_location VARCHAR(100),
    foreign key (user_id) REFERENCES users(user_id)
);
```

#Insert data into partner_preference table

```
INSERT INTO partner_preferences (
preference_id, user_id, preferred_age_from, preferred_age_to, preferred_height_from, preferred_height_to,
preferred_mother_tongue, preferred_religion, preferred_caste, preferred_location
) VALUES
('71000', '1000', 25, 30, 5.50, 6.00, 'Tamil', 'Hindu', 'General', 'Chennai'),
('71001', '1001', 28, 35, 5.60, 6.10, 'Telugu', 'Muslim', 'BC', 'Hyderabad'),
('71002', '1002', 22, 28, 5.40, 5.80, 'Hindi', 'Christian', 'SC', 'Bangalore'),
('71003', '1003', 27, 33, 5.55, 6.00, 'Malayalam', 'Hindu', 'ST', 'Kochi'),
('71004', '1004', 24, 32, 5.45, 5.90, 'Kannada', 'Muslim', 'General', 'Mysore'),
('71005', '1005', 26, 34, 5.50, 6.00, 'Marathi', 'Hindu', 'OBC', 'Pune'),
('71006', '1006', 29, 37, 5.60, 6.05, 'Punjabi', 'Sikh', 'General', 'Amritsar'),
('71007', '1007', 25, 30, 5.50, 6.10, 'Bengali', 'Christian', 'General', 'Kolkata'),
('71008', '1008', 23, 30, 5.60, 6.00, 'Gujarati', 'Hindu', 'BC', 'Ahmedabad'),
('71009', '1009', 28, 36, 5.65, 6.10, 'Urdu', 'Muslim', 'OBC', 'Delhi'),
('71010', '1010', 24, 31, 5.55, 5.90, 'Tamil', 'Hindu', 'General', 'Chennai'),
('71011', '1011', 26, 33, 5.50, 6.00, 'Telugu', 'Christian', 'General', 'Hyderabad'),
('71012', '1012', 30, 38, 5.60, 6.10, 'Hindi', 'Muslim', 'BC', 'Mumbai'),
('71013', '1013', 25, 32, 5.45, 5.95, 'Malayalam', 'Sikh', 'OBC', 'Kochi'),
('71014', '1014', 22, 28, 5.50, 5.80, 'Kannada', 'Hindu', 'ST', 'Bangalore'),
('71015', '1015', 28, 35, 5.60, 6.00, 'Marathi', 'Muslim', 'General', 'Pune'),
```

('71016', '1016', 27, 34, 5.55, 6.00, 'Punjabi', 'Christian', 'General', 'Chandigarh'),

('71017', '1017', 29, 36, 5.60, 6.05, 'Bengali', 'Hindu', 'General', 'Kolkata'),

('71018', '1018', 24, 32, 5.50, 5.95, 'Gujarati', 'Muslim', 'BC', 'Surat'),

('71019', '1019', 26, 33, 5.50, 6.00, 'Urdu', 'Hindu', 'General', 'Delhi');

#Checking tables and data from tables

show tables;

Tables_in_matrimonysalesanalysis
education_details
family_details
location_details
partner_preferences
sales
users
view_location_match
view_user_payment

Result 1 Read Onl

1 Select the user details and their partner preferences

SELECT u.full_name, u.gender, p.preferred_age_from, p.preferred_age_to, p.preferred_height_from, p.preferred_height_to

FROM users u

JOIN partner_preferences p ON u.user_id = p.user_id;

Result Grid Filter Rows: Export: Wrap Cell Content:

full_name	gender	preferred_age_from	preferred_age_to	preferred_height_from	preferred_height_to
Ravi Kumar	Male	25	30	5.50	6.00
Priya Sharma	Female	28	35	5.60	6.10
Arun Raj	Male	22	28	5.40	5.80
Meena George	Female	27	33	5.55	6.00
Vikram Desai	Male	24	32	5.45	5.90
Anjali Rao	Female	26	34	5.50	6.00
Karthik M	Male	29	37	5.60	6.05
Divya S	Female	25	30	5.50	6.10

Result 2 Read Onl

2 Select users who prefer a partner in the age range of 25-30

SELECT u.full_name, p.preferred_age_from, p.preferred_age_to

FROM users u

JOIN partner_preferences p ON u.user_id = p.user_id

WHERE p.preferred_age_from BETWEEN 25 AND 30;

Result Grid Filter Rows: Export: Wrap Cell Content:

full_name	preferred_age_from	preferred_age_to
Ravi Kumar	25	30
Priya Sharma	28	35
Meena George	27	33
Anjali Rao	26	34
Karthik M	29	37
Divya S	25	30
Sneha Iyer	28	36
Lakshmi Menon	26	33

Result 3 Read Only

3 Group users based on the preferred religion and count how many users prefer each religion

```
SELECT p.preferred_religion, COUNT(u.user_id) AS user_count

FROM users u

JOIN partner_preferences p ON u.user_id = p.user_id

GROUP BY p.preferred_religion;

HAVING COUNT(u.user_id) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

preferred_religion	user_count
Hindu	8
Muslim	6
Christian	4
Sikh	2

Result 4 × Read Only

4 Select the full name of the user, their city, and product type for users who prefer a partner aged 25-30

```
SELECT u.full_name, l.city, s.product_type

FROM users u

JOIN location_details l ON u.user_id = l.user_id

JOIN sales s ON u.user_id = s.user_id

JOIN partner_preferences p ON u.user_id = p.user_id

WHERE p.preferred_age_from BETWEEN 25 AND 30;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	city	product_type
Ravi Kumar	Chennai	Silver
Priya Sharma	Delhi	Gold
Meena George	Kochi	Silver
Anjali Rao	Lucknow	Silver
Karthik M	Amritsar	Platinum
Divya S	Ahmedabad	Gold
Sneha Iyer	Kolkata	Silver
Lakshmi Menon	Chandigarh	Platinum

Result 6 × Read Only

6 Select users who prefer a partner with height between 5.5 and 6.0 feet, and order them by preferred height in ascending order

```
SELECT u.full_name, p.preferred_height_from, p.preferred_height_to

FROM users u

JOIN partner_preferences p ON u.user_id = p.user_id

WHERE p.preferred_height_from BETWEEN 5.5 AND 6.0
```

ORDER BY p.preferred_height_from ASC;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	preferred_height_from	preferred_height_to
Ravi Kumar	5.50	6.00
Anjali Rao	5.50	6.00
Divya S	5.50	6.10
Lakshmi Menon	5.50	6.00
Manoj Mehta	5.50	5.80
Rohit Jain	5.50	5.95
Bhavana K	5.50	6.00
Meena George	5.55	6.00

Result 7 | Read Only

7. Get the first 5 users who prefer a partner with a height between 5.5 and 6.0 feet

SELECT u.full_name, p.preferred_height_from, p.preferred_height_to

FROM users u

JOIN partner_preferences p ON u.user_id = p.user_id

WHERE p.preferred_height_from BETWEEN 5.5 AND 6.0

LIMIT 5;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

full_name	preferred_height_from	preferred_height_to
Ravi Kumar	5.50	6.00
Priya Sharma	5.60	6.10
Meena George	5.55	6.00
Anjali Rao	5.50	6.00
Karthik M	5.60	6.05

Result 8 | Read Only

8 Find the average amount_paid for each product type and their respective cities

SELECT s.product_type, l.city, AVG(s.amount_paid) AS avg_amount_paid

FROM sales s

JOIN location_details l ON s.user_id = l.user_id

GROUP BY s.product_type, l.city;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_type	city	avg_amount_paid
Silver	Chennai	1999.000000
Gold	Delhi	4999.000000
Platinum	Mumbai	7999.000000
Silver	Kochi	9999.000000
Gold	Hyderabad	4999.000000
Silver	Lucknow	1999.000000
Platinum	Amritsar	7999.000000
Gold	Ahmedabad	9999.000000

Result 9 | Read Only

9 Find users who prefer a partner aged between 25 and 30 and have paid an amount greater than the average

SELECT u.full_name, p.preferred_age_from, p.preferred_age_to, s.amount_paid

```
FROM users u
JOIN partner_preferences p ON u.user_id = p.user_id
JOIN sales s ON u.user_id = s.user_id
WHERE p.preferred_age_from BETWEEN 25 AND 30
AND s.amount_paid > (SELECT AVG(amount_paid) FROM sales);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	preferred_age_from	preferred_age_to	amount_paid
Meena George	27	33	9999.00
Karthik M	29	37	7999.00
Divya S	25	30	9999.00
Lakshmi Menon	26	33	9999.00
Sangeetha D	28	35	9999.00
Bhavana K	26	33	9999.00

Result 10 × Read Only

10 Find users who have paid more than the average amount for their product type using a CTE.

```
WITH AvgPayment AS (
    SELECT product_type, AVG(amount_paid) AS avg_payment
    FROM sales
    GROUP BY product_type
)
SELECT u.full_name, s.product_type, s.amount_paid
FROM users u
JOIN sales s ON u.user_id = s.user_id
JOIN AvgPayment a ON s.product_type = a.product_type
WHERE s.amount_paid > a.avg_payment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	product_type	amount_paid
Meena George	Silver	9999.00
Divya S	Gold	9999.00
Lakshmi Menon	Platinum	9999.00
Sangeetha D	Gold	9999.00
Bhavana K	Silver	9999.00

Result 11 × Read Only

11 User Preferences Matching Their City by using views

```
CREATE VIEW view_location_match AS
SELECT u.user_id, u.full_name, l.city AS user_city,
    p.preferred_location
FROM users u
JOIN location_details l ON u.user_id = l.user_id
JOIN partner_preferences p ON u.user_id = p.user_id
WHERE l.city = p.preferred_location;
```

12 Users with Their Payment and Membership Details

```
CREATE VIEW view_user_payment AS
SELECT
    u.user_id,
    u.full_name,
    u.marital_status,
    s.product_type,
    s.amount_paid,
    s.payment_date
FROM users u
JOIN sales s ON u.user_id = s.user_id;
```

1 Total Revenue by Membership Type

```
SELECT product_type,
        COUNT(*) AS total_sales,
        SUM(amount_paid) AS total_revenue,
        AVG(amount_paid) AS avg_payment
FROM sales
GROUP BY product_type
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_type	total_sales	total_revenue	avg_payment
Platinum	6	49994.00	8332.333333
Gold	7	44993.00	6427.571429
Silver	7	29993.00	4284.714286

Result 12 × Read Only

1 Monthly Sales Trend

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS month,
    COUNT(*) AS total_sales,
    SUM(amount_paid) AS total_revenue
FROM sales
GROUP BY month
ORDER BY month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	total_sales	total_revenue
2023-01	1	1999.00
2023-02	1	4999.00
2023-03	2	17998.00
2023-04	1	4999.00
2023-05	1	1999.00
2023-06	1	7999.00
2023-07	1	9999.00
2023-08	1	4999.00

Result 13 × Read Only

3 Top 5 Paying Users--

```
SELECT u.full_name, s.product_type, s.amount_paid, s.payment_date
FROM sales s
JOIN users u ON s.user_id = u.user_id
ORDER BY s.amount_paid DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

full_name	product_type	amount_paid	payment_date
Meena George	Silver	9999.00	2023-03-20
Divya S	Gold	9999.00	2023-07-23
Lakshmi Menon	Platinum	9999.00	2023-11-05
Sangeetha D	Gold	9999.00	2024-01-20
Bhavana K	Silver	9999.00	2024-04-01

Result 14 × Read Only

4 Sales by Location

```
SELECT l.city, l.state, COUNT(*) AS total_sales, SUM(s.amount_paid) AS total_revenue
FROM sales s
JOIN location_details l ON s.user_id = l.user_id
GROUP BY l.city, l.state
```

ORDER BY total_revenue DESC;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	state	total_sales	total_revenue
Kochi	Kerala	1	9999.00
Ahmedabad	Gujarat	1	9999.00
Chandigarh	Chandigarh	1	9999.00
Indore	Madhya Pradesh	1	9999.00
Vadodara	Gujarat	1	9999.00
Mumbai	Maharashtra	1	7999.00
Amritsar	Punjab	1	7999.00
Jaipur	Rajasthan	1	7999.00

Result 15 × Read Only

5 Percentage of Sales by product Type --

```
SELECT product_type,
    COUNT(*) AS total_sales,
    ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM sales), 2) AS percentage
FROM sales
GROUP BY product_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_type	total_sales	percentage
Silver	7	35.00
Gold	7	35.00
Platinum	6	30.00

Result 17 × Read Only

6 Users Paying Above Average for Their product Type-

```
WITH avg_payment AS (
   SELECT product_type, AVG(amount_paid) AS avg_amt
   FROM sales
   GROUP BY product_type
)
SELECT u.full_name, s.product_type, s.amount_paid, a.avg_amt
FROM sales s
JOIN users u ON s.user_id = u.user_id
JOIN avg_payment a ON s.product_type = a.product_type
```

WHERE s.amount_paid > a.avg_amt;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	product_type	amount_paid	avg_amt
Meena George	Silver	9999.00	4284.714286
Divya S	Gold	9999.00	6427.571429
Lakshmi Menon	Platinum	9999.00	8332.333333
Sangeetha D	Gold	9999.00	6427.571429
Bhavana K	Silver	9999.00	4284.714286

Result 18 | Read Only

7 Window Function – Rank Users by Payment

SELECT u.full_name, s.product_type, s.amount_paid,

RANK() OVER (PARTITION BY s.product_type ORDER BY s.amount_paid DESC) AS rank_within_membership

FROM sales s

JOIN users u ON s.user_id = u.user_id;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	product_type	amount_paid	rank_within_membership
Meena George	Silver	9999.00	1
Bhavana K	Silver	9999.00	1
Ravi Kumar	Silver	1999.00	3
Anjali Rao	Silver	1999.00	3
Sneha Iyer	Silver	1999.00	3
Suresh Babu	Silver	1999.00	3
Abhishek Nair	Silver	1999.00	3
Divya S	Gold	9999.00	1

Result 19 | Read Only

7 Users Who Paid More Than the Average Payment

SELECT full_name, amount_paid

FROM users u

JOIN sales s ON u.user_id = s.user_id

WHERE amount_paid > (

SELECT AVG(amount_paid) FROM sales

);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	amount_paid
Arun Raj	7999.00
Meena George	9999.00
Karthik M	7999.00
Divya S	9999.00
Amitabh Joshi	7999.00
Lakshmi Menon	9999.00
Manoj Mehta	7999.00
Sangeetha D	9999.00

Result 21 | Read Only

9.Users with the Highest Payment in Each product Type

```
SELECT full_name, product_type, amount_paid
FROM users u
JOIN sales s ON u.user_id = s.user_id
WHERE (product_type, amount_paid) IN (
    SELECT product_type, MAX(amount_paid)
    FROM sales
    GROUP BY product_type
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	product_type	amount_paid
Meena George	Silver	9999.00
Divya S	Gold	9999.00
Lakshmi Menon	Platinum	9999.00
Sangeetha D	Gold	9999.00
Bhavana K	Silver	9999.00

Result 22 × Read Only

10 Cities with Total Sales Above the Average Total per City

```
SELECT city, total_city_revenue
FROM (
    SELECT l.city, SUM(s.amount_paid) AS total_city_revenue
    FROM sales s
    JOIN location_details l ON s.user_id = l.user_id
    GROUP BY l.city
) AS city_sales
WHERE total_city_revenue > (
    SELECT AVG(total_amount)
    FROM (
        SELECT SUM(s.amount_paid) AS total_amount
        FROM sales s
```

```
        JOIN location_details l ON s.user_id = l.user_id
        GROUP BY l.city
    ) AS city_avg
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_city_revenue
Mumbai	7999.00
Kochi	9999.00
Amritsar	7999.00
Ahmedabad	9999.00
Jaipur	7999.00
Chandigarh	9999.00
Coimbatore	7999.00
Indore	9999.00

Result 23 | Read Only

11 Users Whose Payment is the Highest Among All Users

```
SELECT full_name, amount_paid
FROM users u
JOIN sales s ON u.user_id = s.user_id
WHERE amount_paid = (
    SELECT MAX(amount_paid) FROM sales
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	amount_paid
Meena George	9999.00
Divya S	9999.00
Lakshmi Menon	9999.00
Sangeetha D	9999.00
Bhavana K	9999.00

Result 24 | Read Only

11 Users Who Paid More Than the Average for Their product Type

```
SELECT full_name, product_type, amount_paid
FROM users u
JOIN sales s ON u.user_id = s.user_id
WHERE amount_paid > (
    SELECT AVG(amount_paid)
    FROM sales s
    WHERE s.product_type = s.product_type
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	product_type	amount_paid
Arun Raj	Platinum	7999.00
Meena George	Silver	9999.00
Karthik M	Platinum	7999.00
Divya S	Gold	9999.00
Amitabh Joshi	Platinum	7999.00
Lakshmi Menon	Platinum	9999.00
Manoj Mehta	Platinum	7999.00
Sangeetha D	Gold	9999.00

Result 25 | Read Only

12 Users Who Bought Platinum But Paid Below the Average of Platinum Members

```
SELECT full_name, amount_paid
FROM users u
JOIN sales s ON u.user_id = s.user_id
WHERE s.product_type = 'Platinum'
  AND s.amount_paid < (
      SELECT AVG(amount_paid)
      FROM sales
      WHERE product_type = 'Platinum'
  );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	amount_paid
Arun Raj	7999.00
Karthik M	7999.00
Amitabh Joshi	7999.00
Manoj Mehta	7999.00
Rohit Jain	7999.00

Result 26 | Read Only

13 List All Product with Their Revenue, and Flag Those Above Overall Avg

```
SELECT product_type,
       total_revenue,
       CASE
          WHEN total_revenue > (SELECT AVG(total_rev) FROM (
              SELECT SUM(amount_paid) AS total_rev
              FROM sales
              GROUP BY product_type
```

```
        ) AS avg_revs)
        THEN 'Above Average'
        ELSE 'Below Average'
    END AS revenue_status
FROM (
    SELECT product_type, SUM(amount_paid) AS total_revenue
    FROM sales
    GROUP BY product_type
) AS _sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_type	total_revenue	revenue_status
Silver	29993.00	Below Average
Gold	44993.00	Above Average
Platinum	49994.00	Above Average

Result 27 × Read Only